Automated Reasoning - Implementation of a solver for the union of the theories of equality, lists and arrays

Francesco Simbola - VR545665

January 2026

Abstract

This report describes the structure and implementation of an **SMT solver** for the *quantifier-free* combination of three theories: equality with uninterpreted functions **(TE)**, non-empty lists (**Tcons**) and arrays without extensionality (**TA**). The solver, written in C++, checks satisfiability of a conjunction of **equalities** and **disequalities** over terms.

I chose not to implement Nelson-Oppen reasoning, which would need multiple solvers sharing equalities. I decided to reduce Tcons and TA constraints directly to TE by instantiating their axioms as explicit equalities and handling the remaining case splits through **branching**.

Some design choices aim at efficiency. Terms are represented as a **maximally-shared DAG** using **hash-consing**, so comparing terms is just comparing **IDs**. The **Union-Find** structure for equivalence classes uses **eager parent updates**, making **find** operations effectively $O(1)$ at the cost of more expensive merges. I apply, during a merge that chooses which class is representative, the **`ccpar`** heuristic so the class with more parent terms wins, which helps during propagation.

The solver accepts input in a subset of **SMT-LIB 2.0** and handles boolean connectives by converting to **disjunctive normal form** (**DNF**) during parsing. Each resulting conjunction is solved independently.

Experimental evaluation on a suite of **27 test cases** that cover *pure equality*, *list operations*, *array read-over-write patterns* and *mixed-theory formulas* and all tests give the correct **SAT** or **UNSAT** result. Most of the runtime is just startup overhead; solving itself takes microseconds. The main limitation is exponential **blow-up** in DNF conversion for disjunction-heavy

formulas and an $O(k^2)$ worst-case complexity in the **`ccpar`** propagation loop for very large equivalence classes.

1 Implementation Details

Terms are represented as nodes in a *Directed Acyclic Graph (DAG)*. Each node stores a function symbol, a list of argument pointers and a unique integer **ID** assigned during parsing.

Parser checks a cache keyed by a signature string of the form `"funcname arg_id1 arg_id2 . . . "` before creating a new term (**Hash-consing**). If the term already exists, the cached pointer is returned. This guarantees *maximal sharing* so syntactically identical terms share the same node.

Term equality reduces to comparing integer **IDs** (effectively $O(1)$). Without hash-consing, structural equality would require recursive traversal of the term tree which becomes expensive for deep terms and repeated comparisons during Congruence Closure propagation.

1.1 Union-Find with Eager Updates

Equivalence classes are managed by a **Union-Find** structure. I implement an *eager* variant: during each union, the parent pointer of every element in the merged class is updated to point to the new representative.

This makes **Find** operations essentially constant-time ($O(1)$) lookups and because **Congruence Closure** performs **find** far more often than **union**. The cost is paid during merges, where the solver iterates over all members of the smaller class; also, an explicit *member list* is maintained for each class.

On merge, the class with the larger ccpar (parent-term) set becomes the representative. After that, Congruence Closure must check all pairs of parent terms for potential new congruences. Solver minimizes updates to **`ccpar`** sets and reduces redundant work in the propagation loop by keeping the representative with more parent.

Disequality constraints are stored as *forbidden pairs* between class representatives. Before any merge, the solver checks the forbidden list; if the merge is forbidden, it reports **UNSAT**.

Conflict detection becomes $O(1)$ per merge attempt with that. Without this mechanism the solver would need to re-check all disequality literals after every merge (up to $O(n)$ per operation).

The core Congruence Closure algorithm maintains a **pending queue** of pairs to merge. After each successful equality merge, it analyzes all pairs of terms in the combined **`ccpar`** set. Two terms are **congruent** if they have the

same function symbol and *pairwise-equivalent* arguments; congruent pairs are enqueued for merging. Checking all pairs in **`ccpar`** is $O(k^2)$, where k is the **`ccpar`** size. For typical inputs ($k < 50$), this cost is negligible. For very large inputs, grouping parents by a *signature* (function symbol + representatives of arguments) would reduce the check to near-linear, but this optimization is not implemented.

1.2 Theory Integration: Handling `select`-over-`store`

When the solver encounters an `select(base, j)` term where `base` is equivalent to some `store(a, i, v)`, it must determine whether the read index j matches the write index i. Since this relationship is often not determined by existing constraints, the solver applies a recursive strategy:

1. It first searches for unprocessed `(select, store)` pairs using the function `find_select_store`, which builds a temporary Congruence Closure to identify equivalences. Once found, the pair is marked as visited to prevent re-processing.

2. **Branch 1 ($i = j$):** the solver adds the equality $i = j$ to the current literal set and replaces all occurrences of the `select` term with the stored value v. It then calls itself recursively on this modified problem.

3. **Branch 2 ($i \neq j$):** the solver adds the disequality $i \neq j$ and replaces the `select` term with `select(a, j)`—a read from the underlying array bypassing the store. It then recurses on this branch.

If either branch returns SAT, the formula is satisfiable. If both branches return UNSAT, the formula is unsatisfiable. The branching depth is bounded by the number of nested select-over-store patterns, which is typically small in practice.

For `Tcons` no branching is needed. The axioms $\text{car}(\text{cons}(x, y)) = x$ and $\text{cdr}(\text{cons}(x, y)) = y$ are unconditional, so the solver simply instantiates them as explicit equalities and passes them to Congruence Closure.

1.3 Project Structure

The solver is organized into five modules plus an entry point:

The separation of Union-Find from Congruence Closure allows reuse and testing in isolation. The theory solver does not modify CC internals but it only calls `merge()` and `add_inequality()`. This structure simplifies debugging so if a test fails, I can trace whether the bug is in Union-Find (wrong

File	Role
`ast.h`	Defines `Symbol`, `Term`, and `Literal` structures. Terms form a DAG with unique IDs.
`union_find.h`	Implements equivalence classes with eager updates, `ccpar` sets and forbidden pairs.
`congruence_closure.h`	Propagates congruences using the pending queue and `ccpar` lookup and uses Union-Find to manage equivalence classes.
`theory_solver.h`	Instantiates Tcons/TA axioms, triggers branching, invokes CC.
`parser.h`	Reads SMT-LIB input, builds the term DAG via hash-consing, converts to DNF.
`main.cpp`	Reads file or stdin, invokes parser and solver, prints SAT/UNSAT.

representatives), CC (missed congruence) or the theory layer (wrong axiom instantiation).

2 Experiments and Results

I test a suite of **27 test cases** organized into four categories (**TE**, **Tcons**, **TA** and **Mixed/All**).

2.1 Analysis

All 27 tests produce the expected results.

Measured runtimes (5–6 ms per test) are dominated by process startup and file I/O rather than by the reasoning itself. For inputs of this size the solving completes in microseconds. Find operations are constant-time due to eager parent updates and the `ccpar` heuristic keeps the propagation loop efficient by minimizing redundant checks.

`Test_stress1` and `test_stress2` are particularly informative. Solver must derive consequences at multiple levels (e.g., $f(a) = f(b)$, then $g(f(a)) = g(f(b))$ and so on) after asserting a single base equality (e.g., $a = b$). The fact that these tests complete confirms that propagation does not get stuck and does not repeat work unnecessarily. The `ccpar` heuristic contributes by selecting as representative the class with more parent terms, which reduces redundant traversals of large parent sets after merge operations.

Test	Theory	Result	Time (ms)	Description
`test_simple_unsat`	TE	UNSAT	3	Transitivity chain violation
`test_tcons_unsat`	Tcons	UNSAT	6	$\text{car}(\text{cons}(x, y)) \neq x$
`test_ta_unsat`	TA	UNSAT	7	$\text{select}(\text{store}(a, i, v), i) \neq v$
`test_complex_unsat`	All	UNSAT	7	Theories combination
`test_simple_sat`	TE	SAT	6	Consistent equality
`test_tcons_sat`	Tcons	SAT	6	Cons without conflict
`test_ta_sat`	TA	SAT	6	Uncorrelated array indices
`test_stress1`	TE	UNSAT	6	$a = b = c = d = e$, $e \neq a$
`test_stress2`	TE	UNSAT	6	$f(f(f(a))) \neq f(f(f(b)))$, $a = b$
`test_stress3`	TA	UNSAT	6	Multiple nested stores
`test_stress4`	Tcons	UNSAT	6	$\text{car}(\text{cdr}(\text{cons}(\ldots)))$
`test_stress5`	TE	UNSAT	6	$f(a, b) = c$, $f(a, b) = d$, $c \neq d$
`test_stress6`	Tcons	UNSAT	6	Identity reconstruction via car / cdr
`test_stress7`	All	UNSAT	6	$\text{car}(\text{select}(\text{store}(a, i, \text{cons}(x, y)), i)) \neq x$
`test_stress8`	TE	UNSAT	6	$a \neq b$, $c = a$, $c = b$
`test_book_example`	TA	UNSAT	6	Bradley & Manna (adapted)
`test_qf_uf_benchmark`	TE	UNSAT	6	SMT-LIB QF_UF format
`test_sat_verify1-3`	Mixed	SAT	6	Non-regression satisfiable formulas
`test_axiom_verify1-7`	Mixed	UNSAT	6–7	Individual axiom verification

Table 1: Test results.

`Test_stress3` and `test_axiom_verify6` involve nested `store` operations and therefore trigger the branching mechanism. Both branches are explored and both lead to UNSAT, confirming that the case split is sound. The branching depth remains small (at most 2 in these tests), with no evidence of exponential blowup.

The mixed-theory tests (`test_stress7`, `test_axiom_verify7`) require the solver to combine array read-over-write reasoning with list projection axioms within a single derivation chain. The correct UNSAT result shows that axiom instantiation and congruence propagation interact correctly across theory boundaries, which was the main challenge in this project.

3 Conclusions

The reduction-based approach proved effective for this project. By instantiating axioms explicitly rather than implementing a message-passing Nelson–Oppen framework, I avoided the bookkeeping complexity of tracking shared

equalities between theory solvers. The trade-off is that eager instantiation creates some terms that may never be used—but for problems of this scale, the overhead is negligible.

I chose eager Union-Find over classic path compression for simplicity, since updating all parent pointers during a merge is easier than implementing full path compression with deferred updates. In practice, this choice turned out to be well-suited to the access pattern of Congruence Closure, where find operations dominate. A more sophisticated implementation might use union-by-rank with path compression, but the performance difference would only matter for much larger inputs.

One limitation is how easily the $O(k^2)$ loop in `ccpar` propagation could become a bottleneck if equivalence classes grow large. For the tests in this suite, k never exceeds a few dozen, but a single large merge in a real-world benchmark could cause noticeable slowdown. Adding signature-based grouping would address this without major architectural changes so that only parents with matching function symbols are compared.

Future Work

Integrating the theory solver into a DPLL(T) framework would eliminate the DNF explosion problem entirely. Rather than eagerly expanding disjunctions the SAT engine would guide the theory solver through the search space incrementally.

The current branching strategy for arrays is naive because it explores both index cases without considering whether existing constraints already imply one outcome. Smarter heuristics could prune irrelevant branches early. Similarly, supporting array extensionality would require detecting when two arrays must be equal, a capability the current design does not provide.

Despite these limitations, the solver achieves its goals and the modular architecture makes these extensions feasible without major restructuring.